

DevOps Real-Time Scenario Series

Scenario 02: Troubleshooting Kubernetes Cluster Node Failures

Overview

Kubernetes nodes can fail in multiple ways — from kubelet or container runtime issues to memory pressure or network unavailability. In this scenario, we simulate and resolve various node failures by stopping critical services and tweaking eviction policies to reflect real-world production troubleshooting.

Real-Time Use Case

A DevOps engineer is alerted that workloads aren't being scheduled, or pods are getting evicted due to node failure or resource pressure. The goal is to diagnose and bring the node back to a healthy state without impacting availability.

Phase 1: Initial Node Health Check

Step 1: List all of the nodes in the cluster using wide output to include additional columns in the output:

[Copy code](#)

```
$ kubectl get nodes -o wide
```

All of the nodes have a **STATUS** of **Ready**. The **Ready** status means a node is healthy and ready to accept pods. You will force a worker node into a different status to experience a type of failure.

2. Connect to one of the worker nodes using SSH:

[Copy code](#)

```
$ ssh -o StrictHostKeyChecking=no worker1
```

3. Check the status of the kubelet running on the worker node:

[Copy code](#)

```
$ sudo systemctl status kubelet
```

© Kranthi Putti

linkedin.com/in/kranthi-putti

```

● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/lib/systemd/system/kubelet.service; enabled; vendor preset: enabled)
  Drop-In: /usr/lib/systemd/system/kubelet.service.d
           └─10-kubeadm.conf
   Active: active (running) since 11:37:15 UTC; 1min 17s ago
     Docs: https://kubernetes.io/docs/
  Main PID: 16213 (kubelet)
    Tasks: 10 (limit: 4580)
  Memory: 28.8M
    CPU: 1.096s
   CGroup: /system.slice/kubelet.service
           └─16213 /usr/bin/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf

Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314662 16213 reconciler_common.go:251]
Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314684 16213 reconciler_common.go:251]
Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314710 16213 reconciler_common.go:251]
Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314747 16213 reconciler_common.go:251]
Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314763 16213 reconciler_common.go:251]
Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314795 16213 reconciler_common.go:251]
Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314828 16213 reconciler_common.go:251]
Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314844 16213 reconciler_common.go:251]
Jun 27 11:37:16 ip-10-0-0-10 kubelet[16213]: I0627 11:37:16.314879 16213 reconciler_common.go:251]

```

Recall that the kubelet is the primary agent running on nodes that watches for pod specs. Ubuntu 22 is a systemd-based Linux operating system. The tool used to manage services in systemd-based systems is systemctl. Common systemctl commands include:

- **status:** This command shows a concise status summary of the **kubelet** service. It includes its current operational state (e.g., **active (running)**), the primary location of its main **systemd** unit file (**/lib/systemd/system/kubelet.service**), and indicates that configuration overrides are applied via 'drop-in' files from the directory **/usr/lib/systemd/system/kubelet.service.d/** (specifically listing **10-kubeadm.conf** file as one of them). It also displays the full command executed to start the service (e.g., **/usr/bin/kubelet --bootstrap-kubeconfig=...**), and typically, the ten most recent log messages are shown at the bottom of the output. This comprehensive information is invaluable for quickly assessing the health and configuration of the **Kubelet** on a node, especially when debugging issues.
- **start:** Starts a service
- **stop:** Stops a service
- **enable:** Enables a service so that it is automatically started on boot. Note that enable does not automatically start a service until the system is restarted. Therefore, this command is usually followed by start to start the service without rebooting. (there is also the **--now** option of enable to both enable and start the service in one command)

4. Press **q** to quit the status output view.

5. Enter the following to view all of the log messages associated with kubelet service:

[Copy code](#)

```
$ journalctl -u kubelet
```

A service is a type of unit in systemd. The `-u` shows messages for a specific unit, in this case the kubelet service. You can search these messages for any errors that occur and possibly put the worker node into a failed state. Next, you simulate a new error.

6. Stop the kubelet running on the worker node:

[Copy code](#)

```
$ sudo systemctl stop kubelet
```

You can confirm the service stopped by viewing the status output again. Another service you could stop to create a failure would be containerd, which is the container runtime used by the nodes in this lab. When diagnosing an outage, you should check the status of both the kubelet and the container runtime.

7. Close the ssh connection to the worker to return to the bastion host's shell:

[Copy code](#)

```
$ exit
```

8. List and watch the status of all the nodes in the cluster:

[Copy code](#)

```
$ watch kubectl get nodes
```

NAME	STATUS	ROLES
ip-10-0-0-10.us-west-2.compute.internal	NotReady	<none>
ip-10-0-0-100.us-west-2.compute.internal	Ready	control-plane
ip-10-0-0-11.us-west-2.compute.internal	Ready	<none>

In a few seconds, notice that the first worker node has a **STATUS** of **NotReady**. Because you created the failure, you know exactly what is wrong. But if it was an unexpected failure, you would issue the `systemctl status` and `journalctl` commands to diagnose the problem.

Press `ctrl+c` to stop watching the nodes.

9. Start the kubelet service on the failed worker node:

[Copy code](#)

```
$ ssh worker1 'sudo systemctl start kubelet'
```

10. List and watch the status of all the nodes in the cluster:

[Copy code](#)

```
$ watch kubectl get nodes
```

All of the nodes should be **Ready** within 40 seconds of the kubelet starting. Nodes can also fail to accept pod requests due to resource pressure. For example, if a node is out of disk

space or running low on memory. You will simulate this situation in the following instructions.

Press *ctrl+c* to stop watching the nodes.

11. Restart the worker node's kubelet with an additional option to make it detect memory pressure:

[Copy code](#)

```
$ ssh worker1 'sudo sed -i "s%\(/usr/bin/kubelet\) %\1 --eviction-hard=memory.available<3.5Gi %" /usr/lib/systemd/system/kubelet.service.d/10-kubeadm.conf && sudo systemctl daemon-reload && sudo systemctl restart kubelet'
```

This forces the kubelet to report memory pressure on systems with <3.5Gi available.

The commands add the `--eviction-hard` option to the command that starts the kubelet in `/usr/lib/systemd/system/kubelet.service.d/10-kubeadm.conf`. The `--eviction-hard` sets the threshold for when Kubernetes starts to detect memory pressure and is forced to start evicting pods to try to reclaim memory. Its value is 100 mebibytes (Mi) by default. The node only has 3.8Gi of memory in total. Setting the threshold to 3.5Gi means Kubernetes will always detect memory pressure.

12. List the status of all the nodes in the cluster:

[Copy code](#)

```
$ kubectl get nodes
```

All of the nodes still have a **STATUS** of **Ready**. However, if you were scheduling pods, you may notice that one worker is not accepting any new pods and is evicting existing pods. This requires more details to diagnose than the `get` command provides.

13. Describe the worker node with the increased hard eviction threshold:

[Copy code](#)

```
$ kubectl describe node ip-10-0-0-10 | more
```

Focus on the **Conditions** section near the top of the output:

Conditions:	
Type	Status
----	-----
NetworkUnavailable	False
MemoryPressure	True
DiskPressure	False
PIDPressure	False
Ready	True

You may notice the node shows **MemoryPressure**. This means it doesn't have enough free memory to schedule new pods, even though its status is still Ready.

If Kubernetes pods are using too much memory, it may evict some to free up space. But if memory is being used by processes **outside Kubernetes**, you'll need to SSH into the node and stop them manually.

Similar issues can happen with low disk space (**DiskPressure**) or too many running processes (**PIDPressure**). In such cases, you can **cordon** or **drain** the node to stop new pods from being scheduled. Once fixed, you can **uncordon** the node to resume scheduling.

NetworkUnavailable means there's a network issue with the node.

14. Press *spacebar* until you have paged through all of the describe output, and the shell prompt is returned to you.

At the end of the output in the **Events** section, which includes the most recent events at the end, you will see the following two events related to the memory pressure:

```
Normal    NodeHasSufficientMemory
Warning   InvalidDiskCapacity
```

15. Restart the worker node's kubelet without the hard eviction limit option to repair the node:

[Copy code](#)

```
$ ssh worker1 'sudo sed -i "s/--eviction-hard=memory.available<3.5Gi %%"
/usr/lib/systemd/system/kubelet.service.d/10-kubeadm.conf && sudo systemctl daemon-
reload && sudo systemctl restart kubelet'
```

16. Confirm that the node no longer reports memory pressure:

[Copy code](#): `$ kubectl describe node ip-10-0-0-10 | more`

```
Conditions:
  Type              Status
  ----              -
  NetworkUnavailable False
  MemoryPressure    False
  DiskPressure       False
  PIDPressure        False
  Ready             True
```

Summary

In this you learned how to detect and resolve several node-level issues that can cause failures or have an undesirable impact on the performance of your Kubernetes cluster.

© Kranthi Putti

linkedin.com/in/kranthi-putti